

# Μεταγλωττιστές 2025-26

Η γραμματική των αριθμητικών εκφράσεων  
(και δύο εντολών)

# Ο στόχος

```
a = 2 + 7.55*44
print a
b = 3*(a-99.01)
print b*0.23
c = 5-3-2
print c
```

- Κατασκευή γραμματικής και συντακτικού αναλυτή LL(1) για αριθμητικές εκφράσεις με μεταβλητές
  - Με τους τελεστές + - \* /
- Και δύο εντολές (statements)
  - ανάθεσης σε μεταβλητές (=)
  - εκτύπωσης (print)

# Χαρακτηριστικά γλώσσας αριθμητικών εκφράσεων

- 2 εντολές (statements)

= (assignment)

print (output)

- 1 τύπος δεδομένων

float

- Χρήση απλών (scalar) μεταβλητών

Δυναμικά, χωρίς δήλωση

Σφαιρική εμβέλεια (global scope)

- Οι βασικοί αριθμητικοί τελεστές

+ - \* /

# Γραμματική LL(1) αριθμητικών εκφράσεων

`Stmt_list`      $\rightarrow$  `Stmt Stmt_list` |  $\epsilon$

- **Stmt\_list**: το αρχικό μη τερματικό σύμβολο
  - Παράγει μια ακολουθία εντολών (statements)  
*Stmt Stmt Stmt Stmt ... Stmt (EndOfText)*

`Stmt`              $\rightarrow$  `id = Expr` | `print Expr`

- **Stmt**: τα δύο είδη εντολών (statements)
  - **Ανάθεση** τιμής αριθμητικής έκφρασης σε μεταβλητή (`id = Expr`)
  - **Εκτύπωση** τιμής αριθμητικής έκφρασης (`print Expr`)

# Αριθμητικές εκφράσεις

- Οι κανόνες της γραμματικής πρέπει να αποτυπώνουν την **προτεραιότητα** των τελεστών σε **3 επίπεδα**
  - από το **χαμηλότερο** στο **υψηλότερο**:
    - + και –
    - \* και /
  - έκφραση σε παρενθέσεις, μεταβλητές, αριθμητικές σταθερές

# Αριθμητικές εκφράσεις

- Έχουμε ήδη δει κάποιες γραμματικές για αριθμητικές εκφράσεις π.χ.

$\text{Expr} \rightarrow \text{Expr} + \text{Term} \mid \text{Expr} - \text{Term} \mid \text{Term}$

$\text{Term} \rightarrow \text{Term} * \text{Factor} \mid \text{Term} / \text{Factor} \mid \text{Factor}$

$\text{Factor} \rightarrow ( \text{Expr} ) \mid \text{id} \mid \text{number}$

- Μπορούμε να χρησιμοποιήσουμε την πιο πάνω αριθμητική;

# Αριθμητικές εκφράσεις

- Έχουμε ήδη δει κάποιες γραμματικές για αριθμητικές εκφράσεις π.χ.

$\text{Expr} \rightarrow \text{Expr} + \text{Term} \mid \text{Expr} - \text{Term} \mid \text{Term}$

$\text{Term} \rightarrow \text{Term} * \text{Factor} \mid \text{Term} / \text{Factor} \mid \text{Factor}$

$\text{Factor} \rightarrow ( \text{Expr} ) \mid \text{id} \mid \text{number}$

- Μπορούμε να χρησιμοποιήσουμε την πιο πάνω αριθμητική;
- Όχι! Η γραμματική αυτή δεν είναι LL(1)!
  - Αριστερή αναδρομή
  - Κοινός παράγοντας στους εναλλακτικούς κανόνες

# Γραμματική LL(1) αριθμητικών εκφράσεων

$\text{Expr} \rightarrow \text{Term Term\_tail}$

$\text{Term\_tail} \rightarrow \text{Addop Term Term\_tail} \mid \varepsilon$

- **Expr** και **Term\_tail**: παράγει ακολουθία όρων με τα + και − ως τελεστές

$\text{Term} +\acute{\eta}- \text{Term} +\acute{\eta}- \text{Term} +\acute{\eta}- \dots +\acute{\eta}- \text{Term}$

$\text{Term} \rightarrow \text{Factor Factor\_tail}$

$\text{Factor\_tail} \rightarrow \text{Multop Factor Factor\_tail} \mid \varepsilon$

- **Term** και **Factor\_tail**: παράγει ακολουθία όρων με τα \* και / ως τελεστές

$\text{Factor} *\acute{\eta}/ \text{Factor} *\acute{\eta}/ \text{Factor} *\acute{\eta}/ \dots *\acute{\eta}/ \text{Factor}$

$\text{Factor} \rightarrow (\text{Expr}) \mid \text{id} \mid \text{number}$

- **Factor**: παράγει τα δομικά στοιχεία των εκφράσεων

έκφραση σε παρενθέσεις, μεταβλητή ή αριθμητική σταθερά



# Η πλήρης γραμματική LL(1)

```
Stmt_list  → Stmt Stmt_list | ε
Stmt       → id = Expr | print Expr
Expr       → Term Term_tail
Term_tail  → Addop Term Term_tail | ε
Term       → Factor Factor_tail
Factor_tail → Multop Factor Factor_tail | ε
Factor     → (Expr) | id | number
Addop      → + | -
Multop     → * | /
```

- Παρατηρήσεις:
  - Δεν υποστηρίζεται το **μοναδιαίο μείον** (-x)
  - Λόγω της μεθόδου συντακτικής ανάλυσης υπάρχει **πρόβλημα προσηταιριστικότητας (associativity)** των τελεστών – και /

| Σύνολα FOLLOW                  | Σύνολα FIRST      | Κανόνες     |                                    |
|--------------------------------|-------------------|-------------|------------------------------------|
| #                              | id, print<br>ε    | Stmt_list   | → Stmt Stmt_list<br>  ε            |
| id, print, #                   | id<br>print       | Stmt        | → id = Expr<br>  print Expr        |
| id, print, ), #                | (, id, number     | Expr        | → Term Term_tail                   |
| id, print, ), #                | +, -<br>ε         | Term_tail   | → Addop Term Term_tail<br>  ε      |
| +, -, id, print, )<br>, #      | (, id, number     | Term        | → Factor Factor_tail               |
| +, -, id, print, )<br>, #      | *, /<br>ε         | Factor_tail | → Multop Factor Factor_tail<br>  ε |
| *, /, +, -, id,<br>print, ), # | (<br>id<br>number | Factor      | → ( Expr )<br>  id<br>  number     |
| (, id, num                     | +<br>-            | Addop       | → +<br>  -                         |
| (, id, num                     | *<br>/            | Multop      | → *<br>  /                         |

# Εκτέλεση εντολών κατά τη διάρκεια της συντακτικής ανάλυσης

- Για την πολύ απλή γλώσσα των αριθμητικών εκφράσεων μπορούμε να εκτελούμε τις εντολές και τις πράξεις **ταυτόχρονα** με τη συντακτική ανάλυση
  - Διερμηνευτής εντολών (interpreter)
  - Η διαδικασία περιγράφεται από τον (γενικότερο) όρο **Syntax Directed Translation**
- **Προσοχή:** Η εκτέλεση εντολών κατά τη διάρκεια της συντακτικής ανάλυσης **δεν είναι η ενδεδειγμένη** όταν η γλώσσα είναι στοιχειωδώς πιο σύνθετη
  - Πώς εκτελούμε υπό συνθήκη;
    - Θα εκτελούμε τον κλάδο που είναι ανενεργός;
  - Πώς χειριζόμαστε τις δομές επανάληψης;
    - Θα αναλύουμε συντακτικά σε κάθε επανάληψη;
  - Τι γίνεται με τις συναρτήσεις;
    - Η συντακτική ανάλυση δεν πρέπει είναι κλήση της συνάρτησης
- Για τον λόγο αυτόν, μετά το απλό σημερινό παράδειγμα θα περάσουμε σε μορφές **ενδιάμεσης αναπαράστασης** και **ξεχωριστής διερμηνείας**

# Η χρήση των μεταβλητών

- Ονόματα μεταβλητών μπορούν να εμφανιστούν
  - Στο αριστερό μέρος (**lhs**) μιας ανάθεσης (=)
  - Μέσα σε κάποια έκφραση (δεξιό μέρος, **rhs**)

**b** = 3\* (**a** - 99.01)

Τυπικά αυτό σημαίνει

**\*(&b)** = 3\* (**\*&(a)** - 99.01)

- rhs: **ανάγνωση** της τιμής από τη διεύθυνση του δεσμευμένου χώρου της μεταβλητής (μια μορφή “dereferencing”)
- lhs: **εγγραφή** νέας τιμής στον δεσμευμένο χώρο της μεταβλητής

# Οι μεταβλητές στην υλοποίησή μας

- Δυναμική χρήση μεταβλητών (χωρίς δήλωση, με έναν μόνο τύπο, **float**)
  - Θα τις τοποθετήσουμε σε ένα **λεξικό** (dict) της Python
    - Το ονομάζουμε **symbol table** (πίνακα συμβόλων)
    - **Κλειδιά** τα ονόματα των μεταβλητών, **τιμές** οι τρέχουσες τιμές των μεταβλητών

```
b = 3*(a-99.01)
```

Στην υλοποίησή μας σημαίνει

```
symbol_table['b'] = 3*(symbol_table['a']-99.01)
```

- rhs: **ανάγνωση** της τιμής της μεταβλητής από το λεξικό
  - θα πρέπει να προηγηθεί έλεγχος ύπαρξης τιμής για τη μεταβλητή  

```
if 'b' in symbol_table:
```

```
    current_value = symbol_table['b']
```
- lhs: **εγγραφή** νέας τιμής της μεταβλητής στο λεξικό
  - είτε υπάρχει προηγούμενη τιμή είτε όχι

# Μέθοδος υλοποίησης

- Η μέθοδος **Expr()** επιστρέφει την τελική τιμή κάθε αριθμητικής έκφρασης
  - Η εντολή ανάθεσης (**=**) τοποθετεί την τελική αυτή τιμή σε κάποια μεταβλητή
  - Η εντολή εξόδου (**print**) τυπώνει την τελική τιμή
- Οι υπόλοιπες μέθοδοι των μη τερματικών συμβόλων κάτω από την Expr() επιστρέφουν την **υπολογισμένη τιμή μέρους της αριθμητικής έκφρασης ή πληροφορία για τον υπολογισμό της**
  - Factor()
  - Addop(), Multop()
  - Factor\_tail(), Term\_tail()
  - Term(), Expr()

# Factor()

- Η μέθοδος **Factor()** επιστρέφει τις πρώτες αριθμητικές τιμές
  - Σε περίπτωση **number** επιστρέφει float(lexeme)
  - Σε περίπτωση **id** επιστρέφει την τρέχουσα τιμή της μεταβλητής από το symbol table
    - Εάν δεν υπάρχει η μεταβλητή στο symbol table θα πρέπει να παράγεται σφάλμα (π.χ. runtime error, uninitialized variable)
  - Στην περίπτωση του **( Expr )** επιστρέφει ό,τι επιστρέφει η κλήση της Expr()

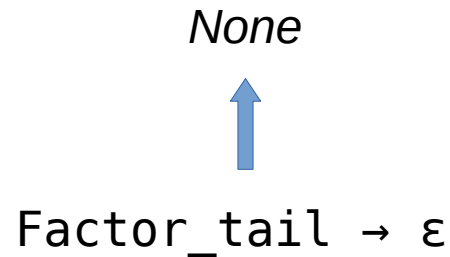
# Addop() και Multop()

- Επιστρέφουν τον τελεστή που αναγνωρίστηκε
  - '+' ή '-' για την Addop()
  - '\*' ή '/' για την Multop()
- Οι Addop() και Multop() δεν επιστρέφουν αριθμητική τιμή αλλά πληροφορία που θα χρειαστεί στις μεθόδους που τις έχουν καλέσει

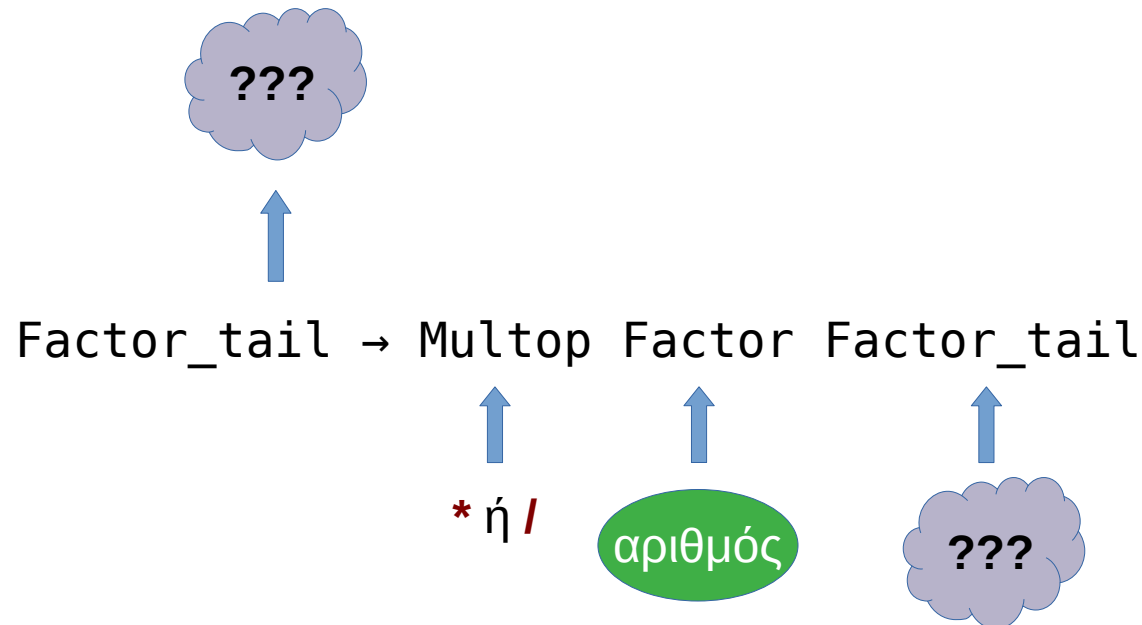


# Factor\_tail()

- Η μέθοδος **Factor\_tail()** επιστρέφει **None** στην περίπτωση της κενής παραγωγής

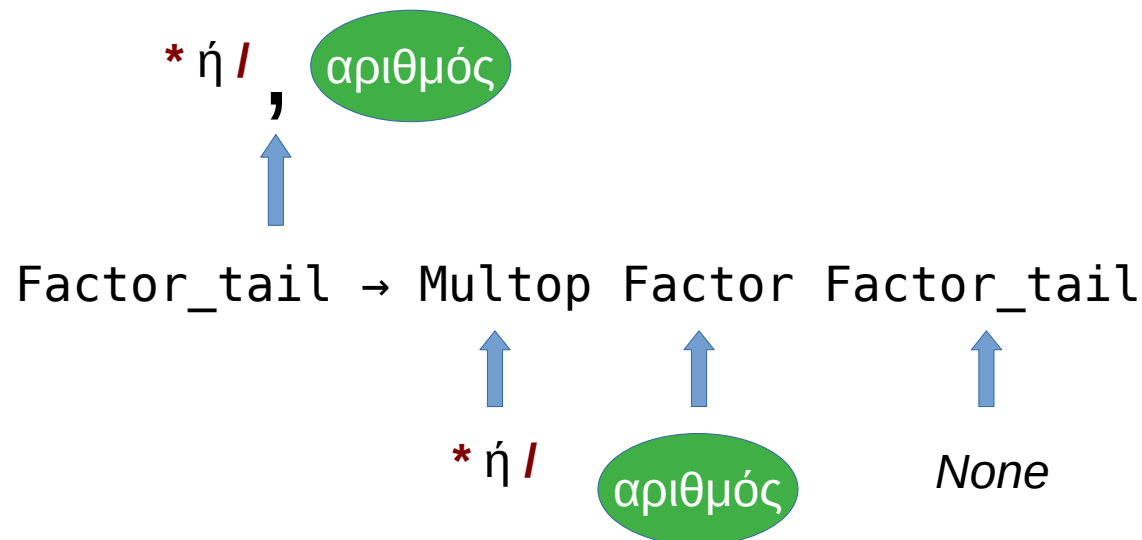


- Τι επιστρέφει όμως στην αντίθετη περίπτωση;



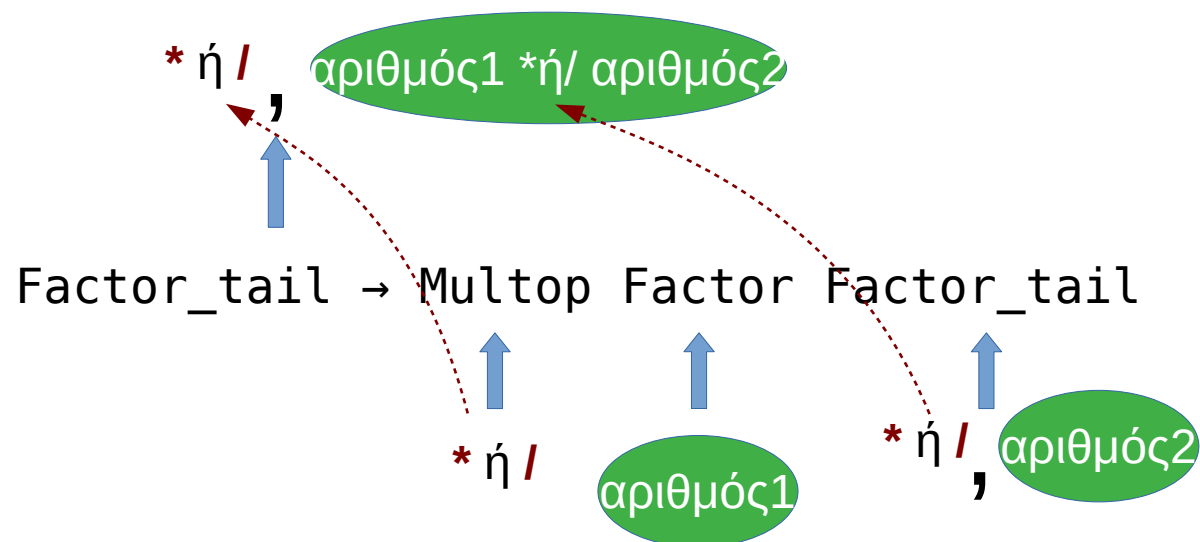
# Factor\_tail(): περίπτωση 1

- **Περίπτωση 1:** Η αναδρομική κλήση της `Factor_tail()` επιστρέφει **None**
- Η διαθέσιμη πληροφορία είναι ο **τελεστής** (`*` ή `/`) και το **δεξιό μέρος** της έκφρασης που εφαρμόζεται στον τελεστή
  - Λείπει το αριστερό μέρος
- Δεν μπορεί να γίνει κάποιος υπολογισμός, το ζευγάρι (**τελεστής, δεξιό μέρος**) επιστρέφεται ως έχει



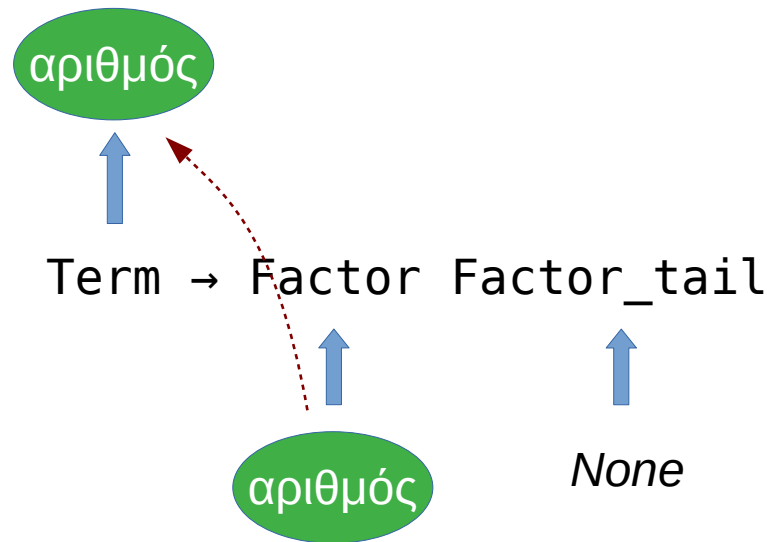
# Factor\_tail(): περίπτωση 2

- Περίπτωση 2: Η αναδρομική κλήση της `Factor_tail()` επιστρέφει ένα ζευγάρι (τελεστής2, αριθμός2)
- Υπολογίζεται η πράξη μεταξύ των αριθμών 1 και 2 με τον τελεστή2 και επιστρέφεται το ζευγάρι (τελεστής1, αριθμός1 \*ή/ αριθμός2)
- Η `Term_tail()` υλοποιείται ακριβώς όπως η `Factor_tail()`, χρησιμοποιώντας όμως το + ή -



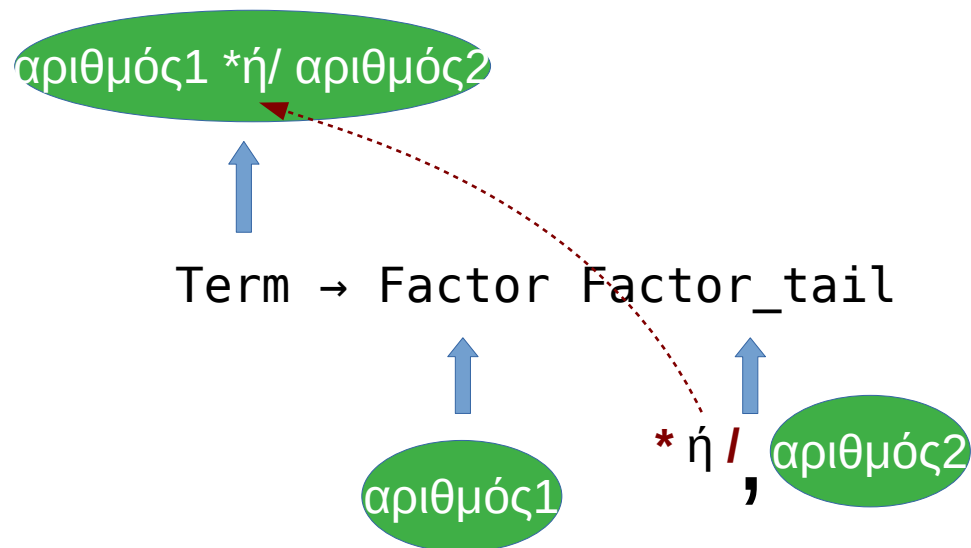
# Term(): περίπτωση 1

- Περίπτωση 1: Η αναδρομική κλήση της `Factor_tail()` επιστρέφει `None`
- Η `Term()` επιστρέφει τον αριθμό που έλαβε μέσω της κλήσης της `Factor()`



# Term(): περίπτωση 2

- **Περίπτωση 2:** Η αναδρομική κλήση της **Factor\_tail()** επιστρέφει ένα ζευγάρι **(τελεστής, αριθμός2)**
- Υπολογίζεται και επιστρέφεται το αποτέλεσμα της πράξης μεταξύ των αριθμών 1 και 2 με τον τελεστή
- Η **Expr()** υλοποιείται ακριβώς όπως η **Term()**, χρησιμοποιώντας αυτή τη φορά το **+** ή **-**



# Το πρόβλημα της προσεταιριστικότητας (associativity) των τελεστών

- Η κλασσική υλοποίηση αναδρομικής κατάβασης **δεν χειρίζεται σωστά** εκφράσεις όπως **5 - 3 - 2 ή 20 / 4 / 2**
  - Οι τελεστές - και / είναι αριστερά προσεταιριστικοί (left associative)
  - Οι πιο πάνω εκφράσεις **πρέπει να υπολογιστούν** ως **(5 - 3) - 2 και (20 / 4) / 2**
- Αντιθέτως, ο συντακτικός αναλυτής τις χειρίζεται ως **5 - (3 - 2) και 20 / (4 / 2)**
  - λόγω της μεθόδου ανάλυσης: **recursive descent LL(1) με δεξιές αναδρομές**

5 - 3 - 2

Expr  $\rightarrow$  Term Term\_tail

5 - 3 - 2

Term\_tail  $\rightarrow$  Addop Term Term\_tail |  $\epsilon$

- 3 - 2

Term\_tail  $\rightarrow$  Addop Term Term\_tail |  $\epsilon$

- 2

Term\_tail  $\rightarrow$  Addop Term Term\_tail |  $\epsilon$

5 - 3 - 2

Expr → Term Term\_tail

5 - 3 - 2

Term\_tail → Addop Term Term\_tail | ε

- 3 - 2

Term\_tail → Addop Term Term\_tail | ε

- 2

Term\_tail → Addop Term Term\_tail | ε



Επιστρέφει  
None



5 - 3 - 2

Expr → Term Term\_tail

5 - 3 - 2

Term\_tail → Addop Term Term\_tail | ε

- 3 - 2

Term\_tail → Addop Term Term\_tail | ε

- 2

Επιστρέφει  
-, 2

Term\_tail → Addop Term Term\_tail | ε

Επιστρέφει  
None



5 - 3 - 2

Expr → Term Term\_tail

5 - 3 - 2

Term\_tail → Addop Term Term\_tail | ε

- 3 - 2

Επιστρέφει  
-, 1 (=3-2)

Term\_tail → Addop Term Term\_tail | ε

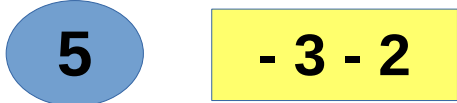
- 2

Επιστρέφει  
-, 2

Term\_tail → Addop Term Term\_tail | ε

Επιστρέφει  
None

5 - 3 - 2

|           |                                                                                       |                           |
|-----------|---------------------------------------------------------------------------------------|---------------------------|
| Expr      | → Term Term_tail                                                                      | Επιστρέφει<br>4 (=5-1)    |
|           |     |                           |
| Term_tail | → Addop Term Term_tail   ε                                                            | Επιστρέφει<br>-, 1 (=3-2) |
|           |     |                           |
| Term_tail | → Addop Term Term_tail   ε                                                            | Επιστρέφει<br>-, 2        |
|           |     |                           |
| Term_tail | → Addop Term Term_tail   ε                                                            | Επιστρέφει<br>None        |
|           |  |                           |

# Διόρθωση σειράς υπολογισμού

- **Πλεονέκτημα** της μεθόδου «αναδρομικής κατάβασης»
  - **Μεγάλη ευελιξία** για τροποποιήσεις του κώδικα
    - Για να εξυπηρετούνται οι ανάγκες μας «**κατά περίπτωση**»
- Μπορεί να αποτελέσει και **μειονέκτημα** όμως...
  - Αποκλίσεις «κατά περίπτωση» από τη «στάνταρντ» μέθοδο δυσκολεύουν την **τεκμηρίωση**
    - Άρα και τη **συντήρηση** του κώδικα

# Τροποποίηση της μεθόδου ανάλυσης

```
Stmt_list  → Stmt Stmt_list | ε
Stmt       → id = Expr | print Expr
Expr       → Term (Addop Term)*
Term_tail → Addop Term Term_tail | ε
Term       → Factor (Multop Factor)*
Factor_tail → Multop Factor Factor_tail | ε
Factor     → (Expr) | id | number
Addop      → + | -
Multop     → * | /
```

- Όσο (while) υπάρχουν τελεστές ίδιου επιπέδου προτεραιότητας, συνεχίζουμε την κατανάλωση εισόδου → επιτυγχάνεται επεξεργασία από αριστερά προς τα δεξιά
  - Το **Term\_tail** «ενσωματώνεται» στο **Expr** και το **Factor\_tail** στο **Term**
  - Επεκταμένη σύνταξη γραμματικής,  $(...)^*$  = «μηδέν ή περισσότερες φορές»

# (Ψευδο)κώδικας για το Expr()

```
def Expr():  
    if next_token in ('(', 'id', 'number'):  
        # Expr → Term (Addop Term)*  
        Term()  
        while next_token in ('+', '-'):  
            Addop()  
            Term()  
    else:  
        raise ParseError
```

Αντίστοιχα για το Term(), με \*,/

- Ο τροποποιημένος κώδικας μπορεί να εξυπηρετήσει τελεστές με **αριστερή μόνο** (-, /) ή με **αριστερή/δεξιά** προσεταιριστικότητα (+, \*)
- Αν υπάρχουν τελεστές με **δεξιά μόνο** προσεταιριστικότητα (π.χ. η ύψωση σε δύναμη) θα πρέπει να χρησιμοποιηθεί **ο αρχικός κώδικας**